МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу «Операционные системы»

Выполнил: А. В. Клюкин
Группа: М8О-201БВ-25
Преподаватель: Е. С. Миронов

Москва, 2026

Условие

Цель работы: Приобретение практических навыков управления процессами в ОС Linux (`fork`, `exec`, `waitpid`), организации обмена данными между процессами через однонаправленные неименованные каналы (`pipe`) и перенаправления стандартных потоков ввода-вывода (`dup2`).

Задание: Составить и отладить программу на языке Си, осуществляющую работу с процессами и межпроцессное взаимодействие в ОС Linux. Основной (родительский) процесс должен создать для решения задачи два дочерних процесса. Взаимодействие между процессами организуется через каналы (`pipe`). Программа должна корректно обрабатывать системные ошибки и освобождать используемые дескрипторы.

Вариант: 21 (Группа вариантов 5).

- Родительский процесс создает два дочерних процесса (`child1` и `child2`), запускающих отдельную программу.
- Первой строкой пользователь в консоль родительского процесса вводит имя файла для `child1`, второй строкой — имя файла для `child2`. Файлы открываются на запись.
- Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в `pipe1` или `pipe2` в зависимости от правила фильтрации.
- **Правило фильтрации:** нечетные строки отправляются в `pipe1`, четные строки — в `pipe2`.
- Дочерние процессы читают строки со стандартного ввода, инвертируют порядок символов в каждой строке и записывают результат в стандартный вывод, перенаправленный в соответствующие выходные файлы.

Метод решения

Программа разделена на две независимые исполняемые единицы: родительский процесс (`parent`) и дочерний процесс (`child`).

1. **Инициализация файлов и каналов:** Родительский процесс считывает с консоли имена двух целевых файлов и открывает их системным вызовом `open()` с флагами `O_WRONLY | O_CREAT | O_TRUNC` и маской прав доступа `0644`. Далее создаются два неименованных канала `pipe1` и `pipe2` с помощью вызова `pipe()`.

2. **Порождение и настройка дочерних процессов:** С помощью `fork()` родитель порождает два дочерних процесса:

   - Для первого процесса (`child1`): через `dup2()` дескриптор чтения `pipe1[0]` подменяет `STDIN_FILENO`, а дескриптор первого файла — `STDOUT_FILENO`.
   - Для второго процесса (`child2`): дескриптор чтения `pipe2[0]` подменяет `STDIN_FILENO`, а дескриптор второго файла — `STDOUT_FILENO`.

   В каждом потомке закрываются все копии исходных дескрипторов, после чего вызывается `execl("./child "child NULL)`.

3. **Передача данных и фильтрация:** Родитель закрывает читающие концы каналов и дескрипторы файлов, после чего входит в цикл чтения пользовательских строк. Чтение произвольной длины реализовано через побайтовый `read()` с динамическим

удвоением размера буфера. Нечетные строки (1, 3, 5...) отправляются в `pipe1[1]`, четные (2, 4, 6...) — в `pipe2[1]`.

4. **Обработка в дочернем процессе:** Каждый дочерний процесс читает строки со своего `STDIN`, инвертирует порядок символов (сохраняя символ переноса строки `\n` на конце) и пишет результат в свой `STDOUT` (в файл).

5. **Завершение работы:** По получении `EOF` родитель закрывает пишущие концы каналов. Дочерние процессы, исчерпав канал, также получают `EOF`, выходят из цикла и корректно завершаются. Родитель вызывает `waitpid()` для обоих потомков, предотвращая появление процессов-зомби.

Описание программы

Проект организован в виде модульной структуры:

- `parent.c` — логика родительского процесса: чтение имен файлов, создание каналов, порождение детей через `fork`, маршрутизация строк и синхронизация завершения через `waitpid`.
- `child.c` — логика дочернего процесса: чтение строк из стандартного ввода, функция `reverse_line` и вывод в стандартный вывод.
- `utils.h` / `utils.c` — вспомогательные функции: `read_line` (динамическое чтение строки с обработкой `EINTR`), `write_all` (гарантированная запись всего буфера), `write_err` и `print_pid`.

Используемые системные вызовы

- `pipe()` — создание однонаправленных каналов межпроцессного обмена.
- `fork()` — создание дочерних процессов путем дублирования вызывающего процесса.
- `dup2()` — перенаправление стандартных дескрипторов ввода и вывода.
- `execl()` — загрузка и запуск бинарного файла `child` в адресное пространство процесса.
- `open()` — открытие выходных файлов на запись с созданием при отсутствии.
- `read()` / `write()` — прямой низкоуровневый ввод-вывод без промежуточной буферизации.
- `close()` — своевременное закрытие дескрипторов для предотвращения утечек и зависания `read()`.
- `waitpid()` — ожидание смены состояния дочерних процессов и сбор кодов завершения.

Результаты

Сборка программы осуществляется с помощью утилиты `make` со строгими флагами компиляции:

```
gcc -Wall -Wextra -Werror -std=c11 -pedantic -Isrc -o parent src/parent.c src/utils.c
gcc -Wall -Wextra -Werror -std=c11 -pedantic -Isrc -o child src/child.c src/utils.c
```

Тестовый запуск программы

Входные данные в терминале:

```
out1.txt
out2.txt
Hello World
Foo Bar
Abcdef
Test
Very long test string 123456789
End of transmission
```

Вывод родительского процесса в консоль (`stderr`):

```
Parent PID: 114626
Enter filename for child1: Enter filename for child2: Enter lines (Ctrl+D to finish):
Child PID: 114627
Child PID: 114628
Parent: done
```

Содержимое первого файла `out1.txt` (строки 1, 3, 5, инвертированные):

```
dlroW olleH
fedcbA
987654321 gnirts tset gnol yreV
```

Содержимое второго файла `out2.txt` (строки 2, 4, 6, инвертированные):

```
raB ooF
tseT
noissimsnart fo dnE
```

Особенности и проверка граничных ситуаций

- Программа корректно обрабатывает строки произвольной длины благодаря динамическому выделению памяти без фиксированного предела в 256 или 4096 байт.
- Символ переноса строки `\n` сохраняется на конце каждой инвертированной строки.
- При передаче пустого ввода программа не падает с ошибками доступа к памяти (`SIGSEGV`) и создает пустые файлы.
- Тестирование под санитайзерами AddressSanitizer и UndefinedBehaviorSanitizer подтвердило полное отсутствие утечек памяти и выходов за границы буферов.

Выводы

В процессе выполнения лабораторной работы я на практике познакомился с управлением процессами и организацией межпроцессного взаимодействия в Linux с помощью системных вызовов `fork`, `pipe`, `dup2` и `exec`. Разобрался с базовой логикой работы неименованных каналов (`pipe`) и закрытием файловых дескрипторов для корректного завершения программы при окончании ввода данных.

Исходная программа

Родительский процесс (`parent.c`)

```c
#define _POSIX_C_SOURCE 200809L

#include "utils.h"

#include <fcntl.h>
#include <sys/wait.h>
#include <errno.h>
#include <string.h>

static char *read_filename(const char *prompt)
{
    write_err(prompt);

    char *line = NULL;
    ssize_t len = read_line(STDIN_FILENO, &line);

    if (len <= 0) {
        free(line);
        return NULL;
    }

    if (line[len - 1] == '\n') {
        line[len - 1] = '\0';
    } else {
        char *tmp = realloc(line, (size_t)len + 1);
        if (!tmp) {
            free(line);
            return NULL;
        }
        line = tmp;
        line[len] = '\0';
    }

    return line;
}

static void safe_close(int *fd)
{
    if (*fd >= 0) {
        close(*fd);
        *fd = -1;
    }
}

int main(void)
{
    print_pid("Parent PID: ");

    char *fname1 = read_filename("Enter filename for child1: ");
    if (!fname1) {
        write_err("parent: failed to read first filename\n");
        return 1;
    }

```

```
55      char *fname2 = read_filename("Enter filename for child2: ");
56      if (!fname2) {
57          write_err("parent: failed to read second filename\n");
58          free(fname1);
59          return 1;
60      }
61
62      int file_fd1 = open(fname1, O_WRONLY | O_CREAT | O_TRUNC, 0644);
63      if (file_fd1 < 0) {
64          write_err("parent: open file1 failed\n");
65          free(fname1); free(fname2);
66          return 1;
67      }
68
69      int file_fd2 = open(fname2, O_WRONLY | O_CREAT | O_TRUNC, 0644);
70      if (file_fd2 < 0) {
71          write_err("parent: open file2 failed\n");
72          close(file_fd1); free(fname1); free(fname2);
73          return 1;
74      }
75
76      free(fname1);
77      free(fname2);
78
79      int pipe1[2] = {-1, -1};
80      int pipe2[2] = {-1, -1};
81
82      if (pipe(pipe1) < 0) {
83          write_err("parent: pipe1 failed\n");
84          close(file_fd1); close(file_fd2);
85          return 1;
86      }
87
88      if (pipe(pipe2) < 0) {
89          write_err("parent: pipe2 failed\n");
90          close(pipe1[0]); close(pipe1[1]);
91          close(file_fd1); close(file_fd2);
92          return 1;
93      }
94
95      pid_t pid1 = fork();
96      if (pid1 < 0) {
97          write_err("parent: fork child1 failed\n");
98          close(pipe1[0]); close(pipe1[1]);
99          close(pipe2[0]); close(pipe2[1]);
100         close(file_fd1); close(file_fd2);
101         return 1;
102     }
103
104     if (pid1 == 0) {
105         if (dup2(pipe1[0], STDIN_FILENO) < 0 || dup2(file_fd1, STDOUT_FILENO) < 0) {
106             write_err("child1: dup2 failed\n");
107             _exit(1);
108         }
109         close(pipe1[0]); close(pipe1[1]);
110         close(pipe2[0]); close(pipe2[1]);
111         close(file_fd1); close(file_fd2);
112         execl("./child", "child", NULL);
```

```
113         write_err("child1: execl failed\n");
114         _exit(1);
115     }
116 
117     pid_t pid2 = fork();
118     if (pid2 < 0) {
119         write_err("parent: fork child2 failed\n");
120         close(pipe1[0]); close(pipe1[1]);
121         close(pipe2[0]); close(pipe2[1]);
122         close(file_fd1); close(file_fd2);
123         waitpid(pid1, NULL, 0);
124         return 1;
125     }
126 
127     if (pid2 == 0) {
128         if (dup2(pipe2[0], STDIN_FILENO) < 0 || dup2(file_fd2, STDOUT_FILENO) < 0) {
129             write_err("child2: dup2 failed\n");
130             _exit(1);
131         }
132         close(pipe1[0]); close(pipe1[1]);
133         close(pipe2[0]); close(pipe2[1]);
134         close(file_fd1); close(file_fd2);
135         execl("./child", "child", NULL);
136         write_err("child2: execl failed\n");
137         _exit(1);
138     }
139 
140     close(pipe1[0]); pipe1[0] = -1;
141     close(pipe2[0]); pipe2[0] = -1;
142     close(file_fd1); file_fd1 = -1;
143     close(file_fd2); file_fd2 = -1;
144 
145     write_err("Enter lines (Ctrl+D to finish):\n");
146 
147     int line_num = 0;
148     int rc = 0;
149 
150     for (;;) {
151         char *line = NULL;
152         ssize_t len = read_line(STDIN_FILENO, &line);
153 
154         if (len < 0) {
155             write_err("parent: read error\n");
156             rc = 1;
157             break;
158         }
159 
160         if (len == 0)
161             break;
162 
163         line_num++;
164         int target = (line_num % 2 != 0) ? pipe1[1] : pipe2[1];
165 
166         if (write_all(target, line, (size_t)len) < 0) {
167             write_err("parent: write to pipe failed\n");
168             free(line);
169             rc = 1;
170             break;
```

```
171         }
172
173         free(line);
174     }
175
176     safe_close(&pipe1[1]);
177     safe_close(&pipe2[1]);
178
179     int status1 = 0, status2 = 0;
180     waitpid(pid1, &status1, 0);
181     waitpid(pid2, &status2, 0);
182
183     if (WIFEXITED(status1) && WEXITSTATUS(status1) != 0)
184         rc = 1;
185     if (WIFEXITED(status2) && WEXITSTATUS(status2) != 0)
186         rc = 1;
187
188     write_err("Parent: done\n");
189     return rc;
190 }
```

Листинг 1: Файл `src/parent.c`

Дочерний процесс (`child.c`)

```
1  #define _POSIX_C_SOURCE 200809L
2
3  #include "utils.h"
4
5  #include <string.h>
6
7  static void reverse_line(char *buf, size_t len)
8  {
9      if (len == 0)
10         return;
11
12     size_t end = (buf[len - 1] == '\n') ? len - 1 : len;
13
14     if (end <= 1)
15         return;
16
17     size_t left = 0, right = end - 1;
18     while (left < right) {
19         char tmp = buf[left];
20         buf[left] = buf[right];
21         buf[right] = tmp;
22         left++;
23         right--;
24     }
25 }
26
27 int main(void)
28 {
29     print_pid("Child PID: ");
30
31     for (;;) {
32         char *line = NULL;
```

```c
        ssize_t len = read_line(STDIN_FILENO, &line);

        if (len < 0) {
            write_err("child: read error\n");
            return 1;
        }

        if (len == 0)
            break;

        reverse_line(line, (size_t)len);

        if (write_all(STDOUT_FILENO, line, (size_t)len) < 0) {
            write_err("child: write error\n");
            free(line);
            return 1;
        }

        free(line);
    }

    return 0;
}
```

Листинг 2: Файл `src/child.c`

Заголовочный файл (`utils.h`)

```c
#ifndef UTILS_H
#define UTILS_H

#include <stdlib.h>
#include <unistd.h>

ssize_t read_line(int fd, char **out);
int write_all(int fd, const char *buf, size_t n);
void write_err(const char *msg);
void print_pid(const char *label);

#endif
```

Листинг 3: Файл `src/utils.h`

Вспомогательные функции (`utils.c`)

```c
#define _POSIX_C_SOURCE 200809L

#include "utils.h"

#include <string.h>
#include <errno.h>

#define INITIAL_BUF 128

ssize_t read_line(int fd, char **out)
{
```

```c
    size_t capacity = INITIAL_BUF;
    size_t length = 0;

    char *buf = malloc(capacity);
    if (!buf)
        return -1;

    for (;;) {
        char ch;
        ssize_t n = read(fd, &ch, 1);

        if (n < 0) {
            if (errno == EINTR)
                continue;
            free(buf);
            return -1;
        }

        if (n == 0)
            break;

        if (length == capacity) {
            size_t new_cap = capacity * 2;
            char *tmp = realloc(buf, new_cap);
            if (!tmp) {
                free(buf);
                return -1;
            }
            buf = tmp;
            capacity = new_cap;
        }

        buf[length++] = ch;

        if (ch == '\n')
            break;
    }

    if (length == 0) {
        free(buf);
        *out = NULL;
        return 0;
    }

    *out = buf;
    return (ssize_t)length;
}

int write_all(int fd, const char *buf, size_t n)
{
    size_t written = 0;
    while (written < n) {
        ssize_t w = write(fd, buf + written, n - written);
        if (w < 0) {
            if (errno == EINTR)
                continue;
            return -1;
        }
```

```
        written += (size_t)w;
    }
    return 0;
}

void write_err(const char *msg)
{
    if (msg)
        write_all(STDERR_FILENO, msg, strlen(msg));
}

void print_pid(const char *label)
{
    pid_t pid = getpid();
    char buf[32];
    int idx = 0;
    char rev[32];
    int ri = 0;

    if (pid == 0) {
        buf[idx++] = '0';
    } else {
        pid_t tmp = pid;
        while (tmp > 0) {
            rev[ri++] = (char)('0' + tmp % 10);
            tmp /= 10;
        }
        for (int i = ri - 1; i >= 0; i--)
            buf[idx++] = rev[i];
    }

    write_err(label);
    write_all(STDERR_FILENO, buf, (size_t)idx);
    write_err("\n");
}
```

Листинг 4: Файл `src/utils.c`

Трассировка системных вызовов (`strace`)

Ниже приведена выжимка ключевых системных вызовов работы программы, зафиксированная с помощью утилиты `strace -f`:

```
39 execve("./parent", ["./parent"], 0x7ffeba9e74a8 /* 134 vars */) = 0
39 write(2, "Parent PID: ", 12) = 12
39 write(2, "39", 2) = 2
39 write(2, "\n", 1) = 1
39 write(2, "Enter filename for child1: ", 27) = 27
39 read(0, "o", 1) = 1
39 read(0, "u", 1) = 1
39 read(0, "t", 1) = 1
39 read(0, "1", 1) = 1
39 read(0, ".", 1) = 1
39 read(0, "t", 1) = 1
39 read(0, "x", 1) = 1
39 read(0, "t", 1) = 1
39 read(0, "\n", 1) = 1
39 write(2, "Enter filename for child2: ", 27) = 27
39 read(0, "o", 1) = 1
39 read(0, "u", 1) = 1
39 read(0, "t", 1) = 1
```

```
|| 39 read(0, "2", 1) = 1
|| 39 read(0, ".", 1) = 1
|| 39 read(0, "t", 1) = 1
|| 39 read(0, "x", 1) = 1
|| 39 read(0, "t", 1) = 1
|| 39 read(0, "\n", 1) = 1
|| 39 openat(AT_FDCWD, "out1.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
|| 39 openat(AT_FDCWD, "out2.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 4
|| 39 pipe2([7, 8], 0) = 0
|| 39 pipe2([9, 10], 0) = 0
|| 39 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x70b99c0dda10) = 40
|| 40 dup2(7, 0) = 0
|| 39 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD , child_tidptr=0x70b99c0dda10) = 41
|| 40 dup2(3, 1) = 1
|| 41 dup2(9, 0) = 0
|| 39 write(2, "Enter lines (Ctrl+D to finish):\n", 32) = 32
|| 41 dup2(4, 1) = 1
|| 39 read(0, "H", 1) = 1
|| 39 read(0, "e", 1) = 1
|| 39 read(0, "l", 1) = 1
|| 39 read(0, "l", 1) = 1
|| 39 read(0, "o", 1) = 1
|| 39 read(0, " ", 1) = 1
|| 39 read(0, "W", 1) = 1
|| 39 read(0, "o", 1) = 1
|| 39 read(0, "r", 1) = 1
|| 39 read(0, "l", 1) = 1
|| 39 read(0, "d", 1) = 1
|| 39 read(0, "\n", 1) = 1
|| 39 write(8, "Hello World\n", 12) = 12
|| 39 read(0, "F", 1) = 1
|| 39 read(0, "o", 1) = 1
|| 39 read(0, "o", 1) = 1
|| 41 execve("./child", ["child"], 0x7fff83940518 /* 134 vars */) = 0
|| 40 execve("./child", ["child"], 0x7fff83940518 /* 134 vars */) = 0
|| 39 read(0, " ", 1) = 1
|| 39 read(0, "B", 1) = 1
|| 39 read(0, "a", 1) = 1
|| 39 read(0, "r", 1) = 1
|| 39 read(0, "\n", 1) = 1
|| 39 write(10, "Foo Bar\n", 8) = 8
|| 39 read(0, "A", 1) = 1
|| 39 read(0, "b", 1) = 1
|| 39 read(0, "c", 1) = 1
|| 39 read(0, "d", 1) = 1
|| 39 read(0, "e", 1) = 1
|| 39 read(0, "f", 1) = 1
|| 39 read(0, "\n", 1) = 1
|| 39 write(8, "Abcdef\n", 7) = 7
|| 39 read(0, "T", 1) = 1
|| 39 read(0, "e", 1) = 1
|| 39 read(0, "s", 1) = 1
|| 39 read(0, "t", 1) = 1
|| 39 read(0, "\n", 1) = 1
|| 39 write(10, "Test\n", 5) = 5
|| 39 read(0, "", 1) = 0
|| 41 write(2, "Child PID: ", 11) = 11
|| 40 write(2, "Child PID: ", 11) = 11
|| 41 write(2, "41", 2) = 2
|| 40 write(2, "40", 2) = 2
|| 41 write(2, "\n", 1) = 1
|| 40 write(2, "\n", 1) = 1
|| 41 read(0, "F", 1) = 1
|| 40 read(0, "H", 1) = 1
|| 41 read(0, "o", 1) = 1
|| 40 read(0, "e", 1) = 1
|| 41 read(0, "o", 1) = 1
|| 40 read(0, "l", 1) = 1
|| 41 read(0, " ", 1) = 1
|| 40 read(0, "l", 1) = 1
```

```
41 read(0, "B", 1) = 1
40 read(0, "o", 1) = 1
41 read(0, "a", 1) = 1
40 read(0, " ", 1) = 1
41 read(0, "r", 1) = 1
40 read(0, "W", 1) = 1
41 read(0, "\n", 1) = 1
40 read(0, "o", 1) = 1
40 read(0, "r", 1) = 1
41 write(1, "raB ooF\n", 8) = 8
40 read(0, "l", 1) = 1
41 read(0, "T", 1) = 1
40 read(0, "d", 1) = 1
41 read(0, "e", 1) = 1
40 read(0, "\n", 1) = 1
41 read(0, "s", 1) = 1
41 read(0, "t", 1) = 1
40 write(1, "dlroW olleH\n", 12) = 12
41 read(0, "\n", 1) = 1
40 read(0, "A", 1) = 1
41 write(1, "tseT\n", 5) = 5
40 read(0, "b", 1) = 1
41 read(0, "", 1) = 0
40 read(0, "c", 1) = 1
40 read(0, "d", 1) = 1
40 read(0, "e", 1) = 1
40 read(0, "f", 1) = 1
40 read(0, "\n", 1) = 1
40 write(1, "fedcbA\n", 7) = 7
39 wait4(40, 0x7fff83940388, 0, NULL) = ? ERESTARTSYS (To be restarted if SA_RESTART is set)
40 read(0, "", 1) = 0
39 wait4(40, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 40
39 wait4(41, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 41
39 write(2, "Parent: done\n", 13) = 13
```

Листинг 5: Ключевые системные вызовы (`strace_short.txt`)